

# Digital Forensics Laboratory

## EXPERIMENT NO: 01

**TITLE:** Comprehensive Digital Evidence Acquisition and Command-Line Analysis using FTK Imager, WinPmem, and Volatility

### 1. AIM

To perform a complete forensic acquisition and analysis lifecycle by imaging non-volatile storage using AccessData FTK Imager, capturing live volatile memory (RAM) using the WinPmem command-line utility, and analyzing the extracted memory dump using Volatility 3 commands via the PassMark Volatility Workbench.

### 2. SOFTWARE / TOOLS REQUIRED

- **Operating System:** Windows OS
- **Forensics Tools:**
  - AccessData FTK Imager (GUI for non-volatile physical drive imaging)
  - WinPmem (winpmem\_mini\_x64\_rc2.exe) (CLI tool for volatile memory capture)
  - Volatility 3 Framework (vol.exe / PassMark Volatility Workbench V3.0)
- **Hardware:** Evidence PC/Workstation, Target Storage Drive (hp v221w 16GB USB Device).

### 3. THEORY

A thorough digital forensic investigation requires collecting both data at rest and data in motion using strict operational procedures.

- **Non-Volatile Memory Acquisition:** AccessData FTK Imager creates mathematically verifiable, bit-by-bit physical images of storage devices (saved as Raw/dd) without altering the original evidence.

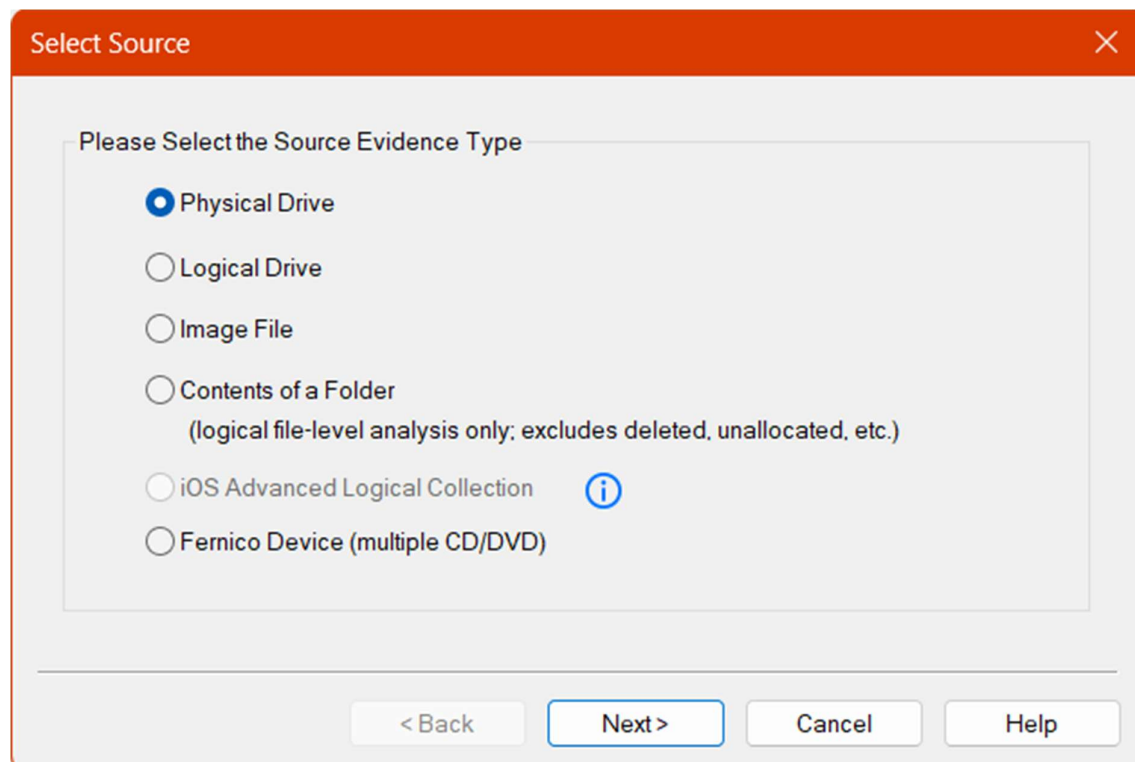
- **Volatile Memory (RAM) Acquisition:** Command-line utilities like **WinPmem** load a specialized kernel driver (pme98A2.tmp) to read physical memory directly. Executing this via CMD writes active processes, network connections, and decrypted keys to a raw dump file (.raw).
- **Volatile Memory Analysis (CLI): Volatility 3** is a framework for analyzing RAM dumps. By passing specific parameters and plugins (e.g., -f <image\_path> windows.pslist.PsList) into the command line or workbench, investigators can reconstruct the system's live state and identify suspicious processes.

## 4. PROCEDURE

### Part A: Acquiring Non-Volatile Memory (Disk Image) using FTK Imager

#### Step 1: Select the Source Type

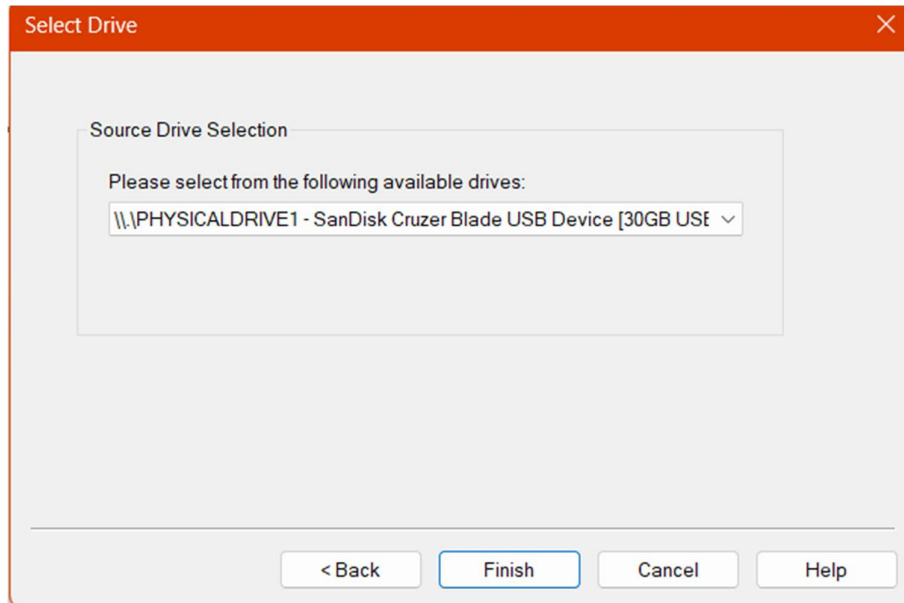
Open FTK Imager as an Administrator. Navigate to **File > Create Disk Image**. In the Select Source dialog, choose **Physical Drive**.



*Figure 1: Selecting Physical Drive as the source evidence type*

## Step 2: Select the Specific Drive

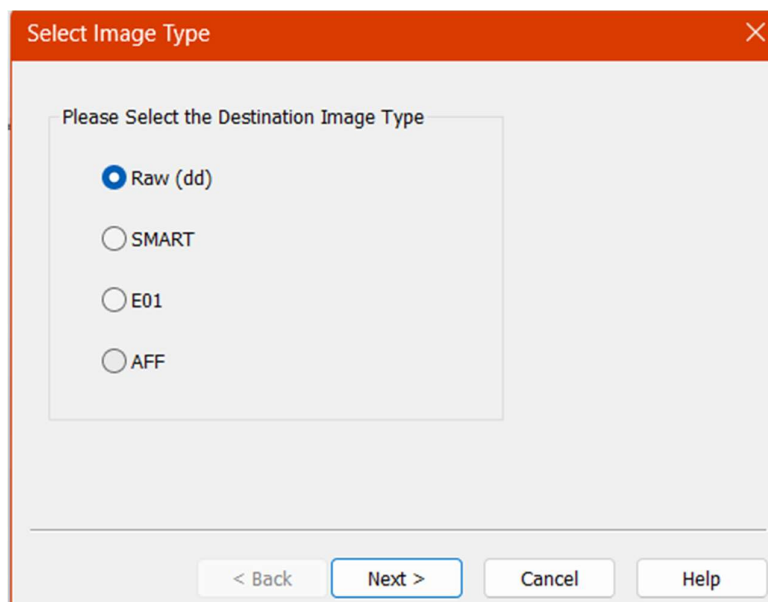
Select the target evidence drive (\\.\PHYSICALDRIVE1 - SanDisk Cruzer Blade USB Device [30GB USB]) from the dropdown.



*Figure 2: Selecting the target USB device for acquisition*

## Step 3: Choose the Image Type

Select **Raw (dd)** as the destination image type to create a pure bitstream copy.

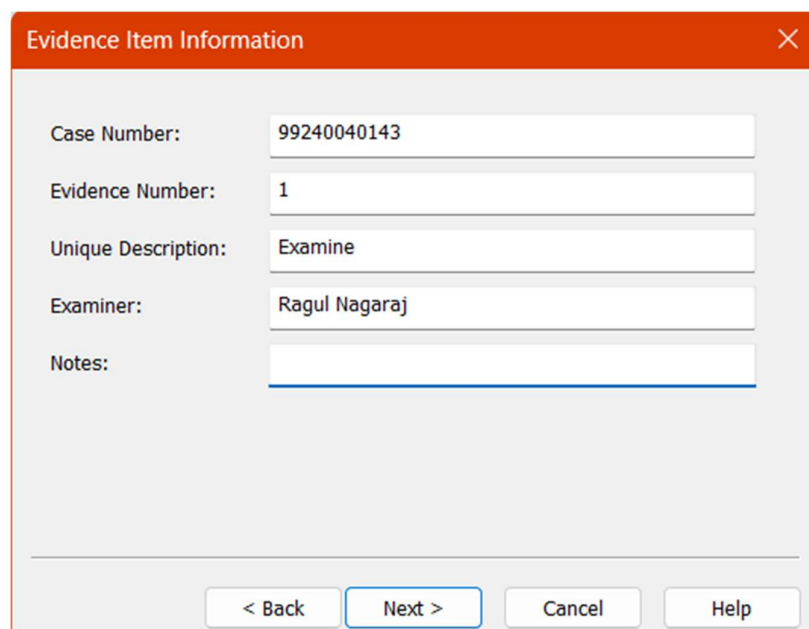


*Figure 3: Configuring the destination image format to Raw (dd)*

#### Step 4: Enter Evidence Item Information

Provide the standardized case details to maintain the chain of custody:

- **Case Number:** 99240040143
- **Evidence Number:** 1
- **Unique Description:** Examine
- **Examiner:** Ragul Nagaraj

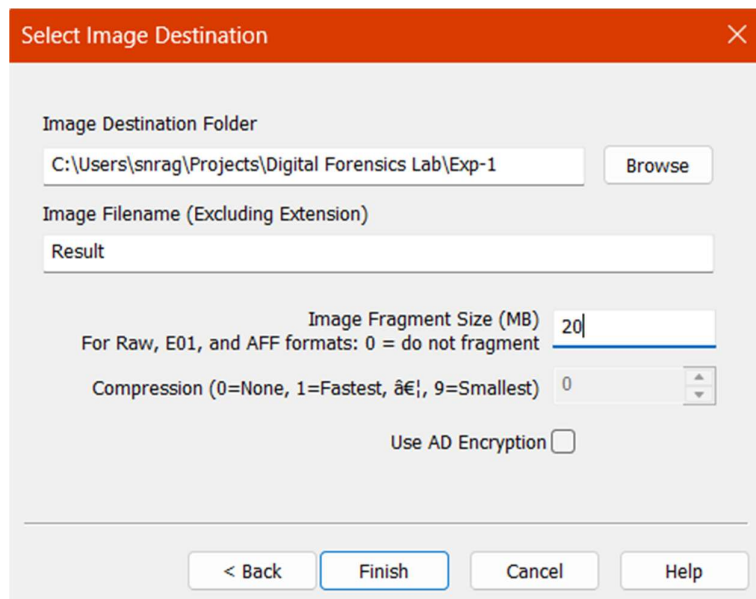


The screenshot shows a dialog box titled "Evidence Item Information" with a close button (X) in the top right corner. The dialog contains five input fields with labels to their left: "Case Number:" with the value "99240040143", "Evidence Number:" with the value "1", "Unique Description:" with the value "Examine", "Examiner:" with the value "Ragul Nagaraj", and "Notes:" with an empty field. At the bottom of the dialog, there are four buttons: "< Back", "Next >" (which is highlighted with a blue border), "Cancel", and "Help".

*Figure 4: Entering the forensic case and examiner details*

#### Step 5: Configure Image Destination

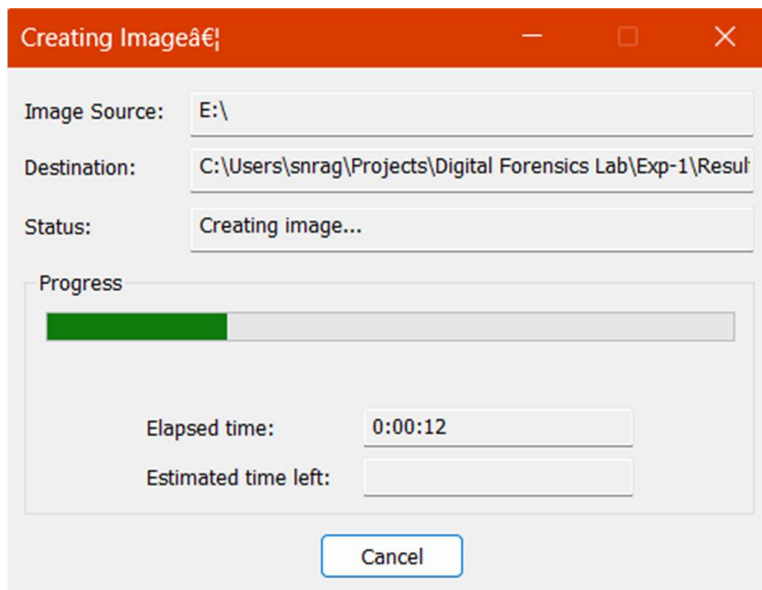
Set the destination folder and the filename to Result. Set the Fragment Size to 20 MB and check **Verify images after they are created.**



*Figure 5: Setting the destination path, filename, and fragmentation size*

#### **Step 6: Monitor Acquisition Progress & Hash Verification**

Click **Start**. Once complete, FTK Imager hashes the original drive and the newly created image. Review the Drive/Image Verify Results.



*Figure 6: Disk imaging progress tracking*

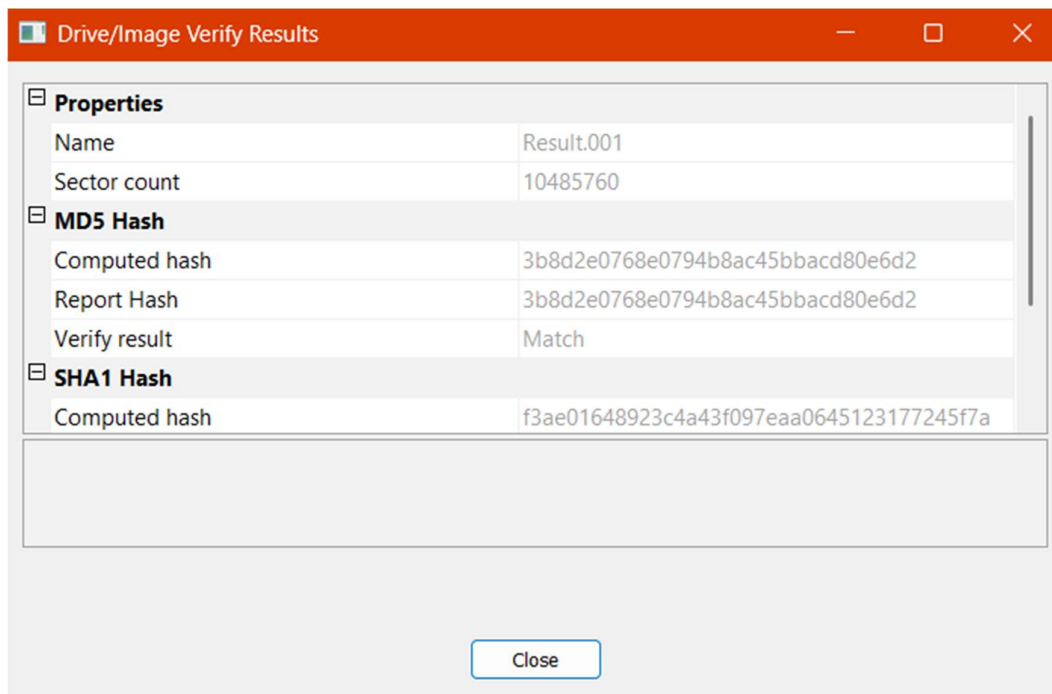


Figure 7: Successful verification of MD5 and SHA1 hashes

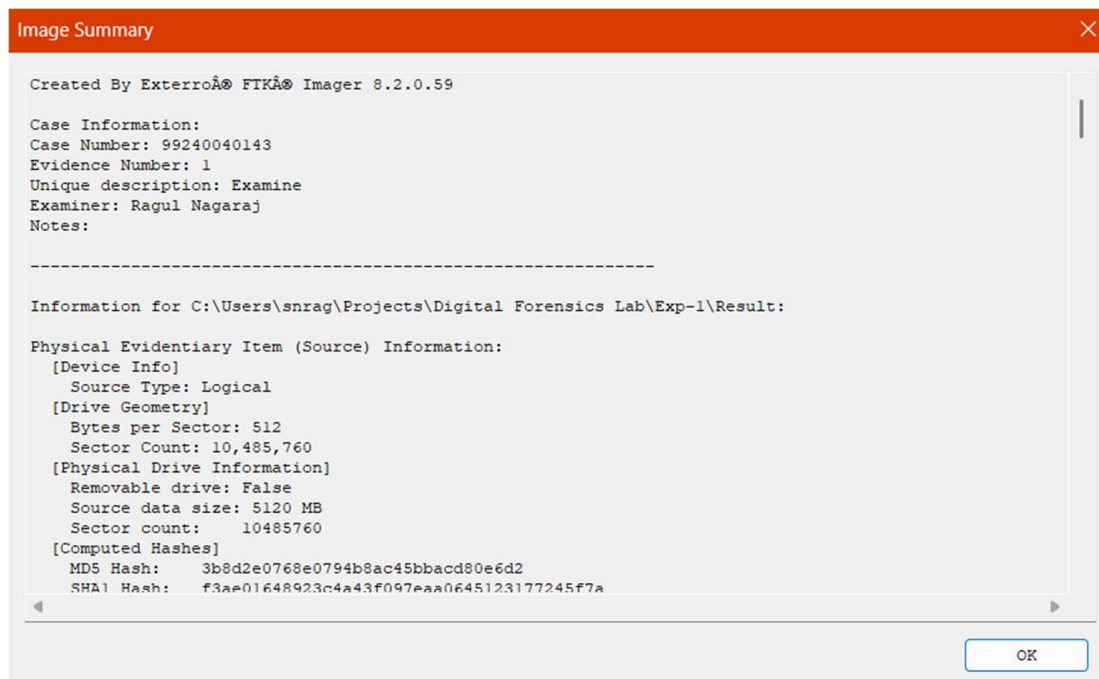
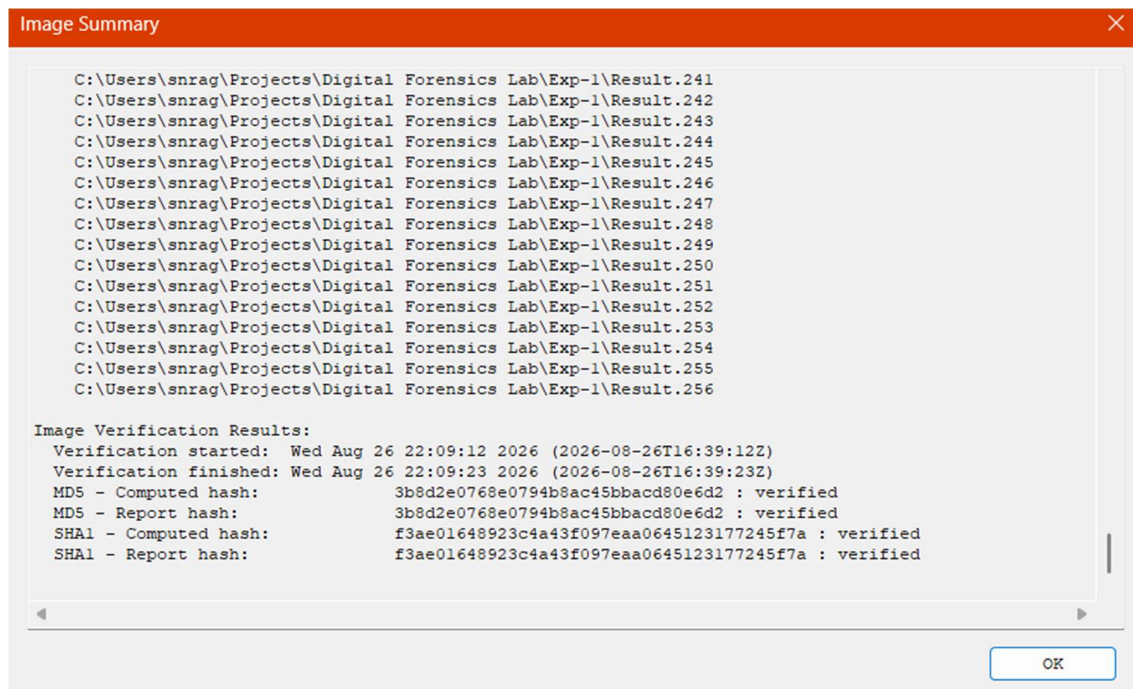


Figure 8: Image Summary detailing the physical device geometry



*Figure 9: Image Summary detailing the hash verification results and segment list*

## **Part B: Acquiring Volatile Memory (RAM) using WinPmem CMD**

### **Step 1: Launch Command Prompt**

Press Win + R, type cmd, and press Ctrl + Shift + Enter to open the Command Prompt as an **Administrator**.

### **Step 2: Navigate to Directory**

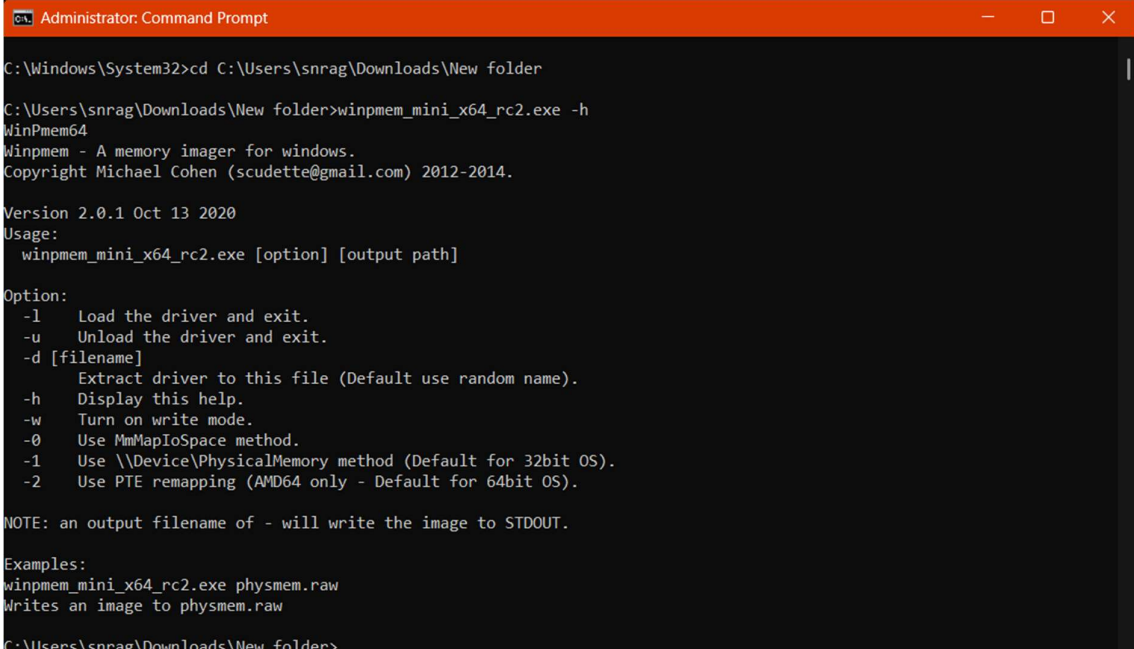
Use the cd command to navigate to the folder containing the WinPmem executable

### Step 3: Check Available Parameters

Execute the help command to view the WinPmem usage parameters:

DOS

winpmem\_mini\_x64\_rc2.exe -h



```
Administrator: Command Prompt
C:\Windows\System32>cd C:\Users\snrag\Downloads\New folder
C:\Users\snrag\Downloads\New folder>winpmem_mini_x64_rc2.exe -h
WinPmem64
WinPmem - A memory imager for windows.
Copyright Michael Cohen (scudette@gmail.com) 2012-2014.

Version 2.0.1 Oct 13 2020
Usage:
  winpmem_mini_x64_rc2.exe [option] [output path]

Option:
  -l   Load the driver and exit.
  -u   Unload the driver and exit.
  -d [filename]
       Extract driver to this file (Default use random name).
  -h   Display this help.
  -w   Turn on write mode.
  -0   Use MmMapIoSpace method.
  -1   Use \\Device\PhysicalMemory method (Default for 32bit OS).
  -2   Use PTE remapping (AMD64 only - Default for 64bit OS).

NOTE: an output filename of - will write the image to STDOUT.

Examples:
winpmem_mini_x64_rc2.exe physmem.raw
Writes an image to physmem.raw
C:\Users\snrag\Downloads\New folder>
```

*Figure 10: Viewing the WinPmem help menu and available parameters*

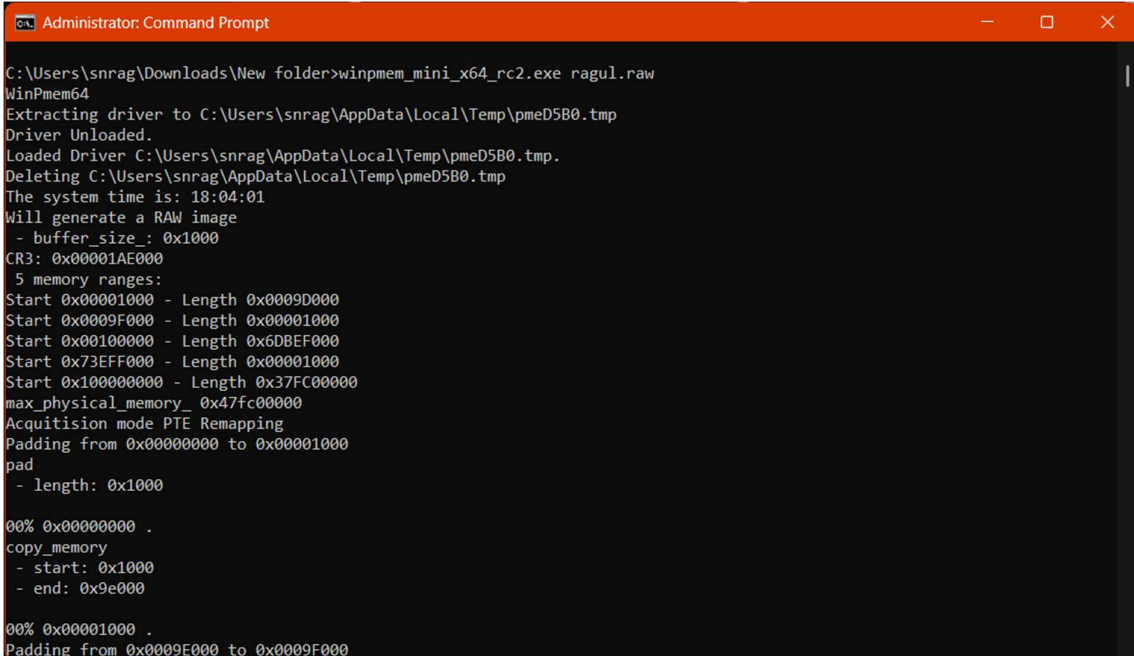


#### Step 4: Execute Memory Capture Command

Run the memory extraction command, specifying the output filename ragul.raw. The utility will map the physical memory and write it to the file:

DOS

winpmem\_mini\_x64\_rc2.exe ragul.raw



```
Administrator: Command Prompt

C:\Users\snrag\Downloads\New folder>winpmem_mini_x64_rc2.exe ragul.raw
WinPmem64
Extracting driver to C:\Users\snrag\AppData\Local\Temp\pmeD5B0.tmp
Driver Unloaded.
Loaded Driver C:\Users\snrag\AppData\Local\Temp\pmeD5B0.tmp.
Deleting C:\Users\snrag\AppData\Local\Temp\pmeD5B0.tmp
The system time is: 18:04:01
Will generate a RAW image
- buffer_size: 0x1000
CR3: 0x00001AE000
5 memory ranges:
Start 0x00001000 - Length 0x0009D000
Start 0x0009F000 - Length 0x00001000
Start 0x00100000 - Length 0x6DBEF000
Start 0x73EFF000 - Length 0x00001000
Start 0x100000000 - Length 0x37FC00000
max_physical_memory_ 0x47fc00000
Acquisition mode PTE Remapping
Padding from 0x00000000 to 0x00001000
pad
- length: 0x1000

00% 0x00000000 .
copy_memory
- start: 0x1000
- end: 0x9e000

00% 0x00001000 .
Padding from 0x0009F000 to 0x0009F000
```

Figure 11: Initializing WinPmem and loading the extraction driver

```
Administrator: Command Prompt - winpmem_mini_x64_rc2.exe ragul.raw
08% 0x64100000 .....
Padding from 0x6DCEF000 to 0x73EFF000
pad
- length: 0x6210000

09% 0x6DCEF000 .....
copy_memory
- start: 0x73eff000
- end: 0x73f00000

10% 0x73EFF000 .
Padding from 0x73F00000 to 0x100000000
pad
- length: 0x8c100000

10% 0x73F00000 .....
10% 0x73F00000 .....
10% 0x73F00000 .....
copy_memory
- start: 0x100000000
- end: 0x47fc00000

22% 0x100000000 .....X.....XXXX.....
26% 0x132000000 x...x.....
30% 0x164000000 .....
30% 0x164000000 .....
```

Figure 12: Memory capture in progress

## Step 5: Verification of Dump

Wait for the terminal to display Driver Unloaded. Navigate to the directory to verify the 18.2 GB ragul.raw file.

```
Administrator: Command Prompt
- length: 0x8c100000

10% 0x73F00000 .....
10% 0x73F00000 .....
10% 0x73F00000 .....
copy_memory
- start: 0x100000000
- end: 0x47fc00000

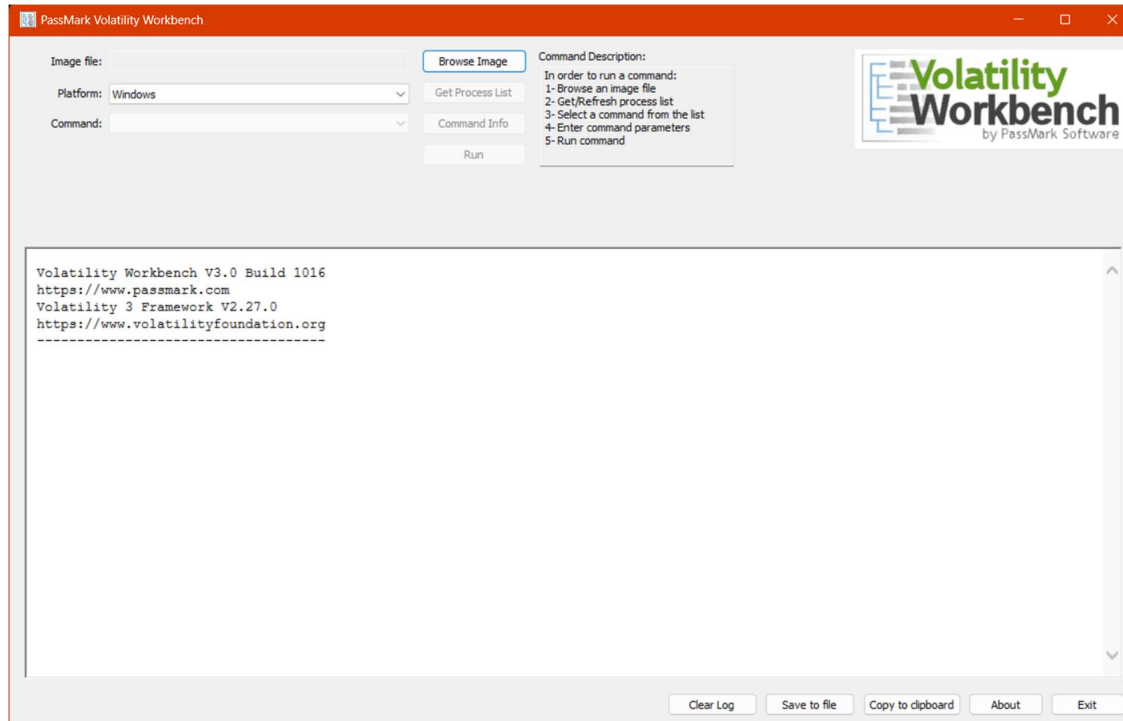
22% 0x100000000 .....X.....XXXX.....
26% 0x132000000 x...x.....
30% 0x164000000 .....
35% 0x196000000 .....
39% 0x1C8000000 .....
43% 0x1FA000000 .....
48% 0x22C000000 .....
52% 0x25E000000 .....
56% 0x290000000 .....
61% 0x2C2000000 .....X.....
65% 0x2F4000000 .....X.....
69% 0x326000000 .....
74% 0x358000000 .....
78% 0x38A000000 .....
83% 0x3BC000000 .....
87% 0x3EE000000 .....
91% 0x420000000 .....
96% 0x452000000 .....
The system time is: 18:12:59
Driver Unloaded.
```

Figure 13: Successful completion of the volatile memory capture

## Part C: Volatile Memory Analysis using Volatility CLI / Workbench

### Step 1: Launch Volatility Environment

Open PassMark Volatility Workbench (which acts as a GUI frontend for vol.exe).



*Figure 16: The default interface of PassMark Volatility Workbench*

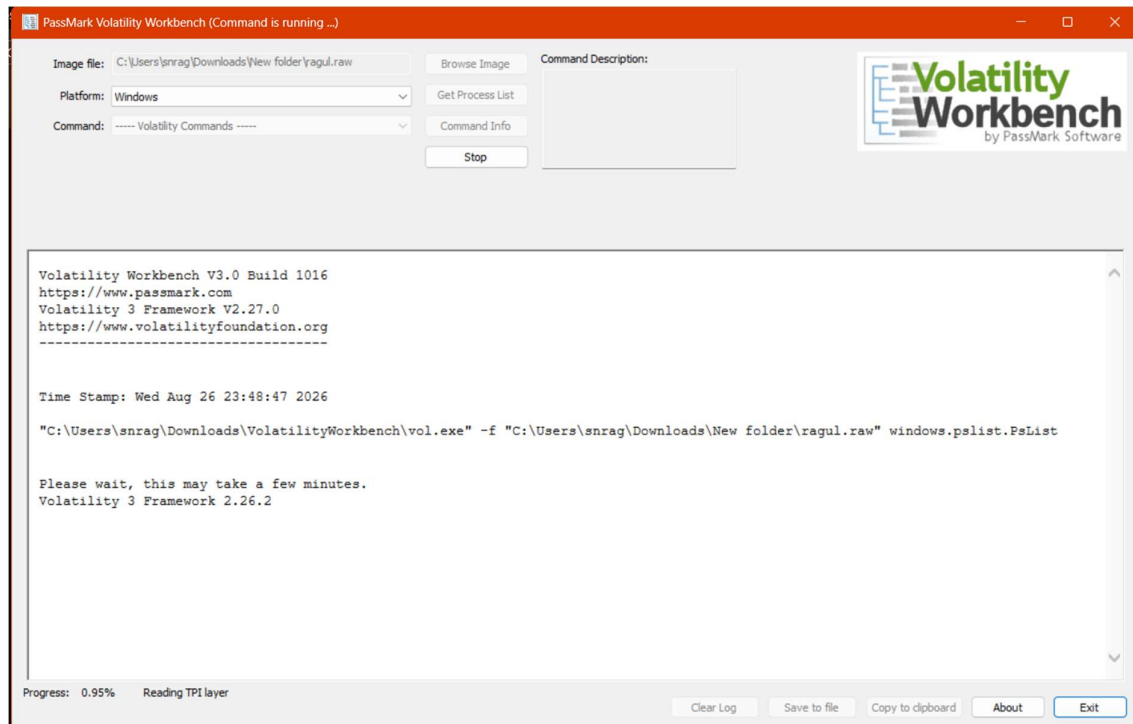
### Step 2: Construct and Execute the Volatility Command

To parse the captured memory and list the active processes, load ragul.raw via **Browse Image**, select the **Windows** platform, choose windows.pslist.PsList from the dropdown, and click **Run**.

The console executes the following command in the background:

DOS

"C:\Users\snrag\Downloads\VolatilityWorkbench\vol.exe" -f C:\Users\snrag\Downloads\New folder\ragul.raw" windows.pslist.PsList (Or) Use the workbench



*Figure 17: Execution of Volatility for the Inba.raw image*

## 5. OBSERVATIONS & RESULTS

### 1. FTK Imager Cryptographic Verification

FTK Imager generated a summary report mathematically confirming the non-volatile image data integrity:

- **MD5 Hash:** 3b8d2e0768e0794b8ac45bbacd80e6d2 : verified
- **SHA1 Hash:** f3ae01648923c4a43f097eaal064512317724517a : verified

### 2. Memory Capture & CLI Parsing

The Inba.raw file was successfully written using the winpmem\_mini\_x64\_rc2.exe command. When analyzed, Volatility parsed the \_EPROCESS blocks in kernel memory. The resulting output log accurately detailed the Process IDs (PID), Parent Process IDs (PPID), Image File Names, and timestamps of all active programs:

Time Stamp: Wed Aug 26 23:48:47 2026

"C:\Users\anrag\Downloads\VolatilityWorkbench\vol.exe" -f "C:\Users\anrag\Downloads\New folder\ragul.raw" windows.plist.PsList

Please wait, this may take a few minutes.

Volatility 3 Framework 2.26.2

| PID  | PPID | ImageFileName   | Offset(V)          | Threads | Handles | SessionId | Wow64                          | CreateTime                     | ExitTime | Disabled | File output |
|------|------|-----------------|--------------------|---------|---------|-----------|--------------------------------|--------------------------------|----------|----------|-------------|
| 4    | 0    | System          | 0xb40c7840c040 412 | -       | N/A     | False     | 2026-08-20 10:33:11.000000 UTC | N/A                            | N/A      | Disabled |             |
| 236  | 4    | Secure System   | 0xb40c786af040 0   | -       | N/A     | False     | 2026-08-20 10:33:08.000000 UTC | N/A                            | N/A      | Disabled |             |
| 280  | 4    | Registry        | 0xb40c786ea040 4   | -       | N/A     | False     | 2026-08-20 10:33:08.000000 UTC | N/A                            | N/A      | Disabled |             |
| 964  | 4    | smss.exe        | 0xb40c95874040 3   | -       | N/A     | False     | 2026-08-20 10:33:11.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1328 | 1112 | csrss.exe       | 0xb40c9f9cc140 15  | -       | 0       | False     | 2026-08-20 10:33:25.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1432 | 1424 | csrss.exe       | 0xb40ca3512140 0   | -       | 1       | False     | 2026-08-20 10:33:28.000000 UTC | 2026-08-20 14:57:38.000000 UTC | Disabled |          |             |
| 1440 | 1112 | wininit.exe     | 0xb40ca3516080 2   | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1508 | 1440 | services.exe    | 0xb40ca363a080 7   | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1528 | 1440 | lsass.exe       | 0xb40ca3636080 1   | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1536 | 1440 | lsass.exe       | 0xb40ca3638080 11  | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1588 | 1424 | winlogon.exe    | 0xb40ca3639080 0   | -       | 1       | False     | 2026-08-20 10:33:28.000000 UTC | 2026-08-20 14:57:37.000000 UTC | Disabled |          |             |
| 1760 | 1508 | svchost.exe     | 0xb40ca369d080 24  | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1772 | 1508 | WUDFHost.exe    | 0xb40ca369e080 5   | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1812 | 1588 | fontdrvhost.exe | 0xb40ca37310c0 0   | -       | 1       | False     | 2026-08-20 10:33:28.000000 UTC | 2026-08-20 14:57:37.000000 UTC | Disabled |          |             |
| 1820 | 1440 | fontdrvhost.exe | 0xb40ca36ca080 5   | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1920 | 1508 | svchost.exe     | 0xb40ca375e080 12  | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 1992 | 1508 | svchost.exe     | 0xb40ca37ef080 6   | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 2020 | 1508 | WUDFHost.exe    | 0xb40ca385c080 20  | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 8    | 1588 | LogonUI.exe     | 0xb40ca3906080 0   | -       | 1       | False     | 2026-08-20 10:33:28.000000 UTC | 2026-08-20 11:35:32.000000 UTC | Disabled |          |             |
| 1084 | 1588 | dmv.exe         | 0xb40ca3905080 0   | -       | 1       | False     | 2026-08-20 10:33:28.000000 UTC | 2026-08-20 14:57:37.000000 UTC | Disabled |          |             |
| 1684 | 1508 | WUDFHost.exe    | 0xb40ca36b1080 11  | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | N/A                            | N/A      | Disabled |             |
| 2040 | 1508 | svchost.exe     | 0xb40ca3963080 0   | -       | 0       | False     | 2026-08-20 10:33:28.000000 UTC | 2026-08-20 10:35:29.000000 UTC | Disabled |          |             |

Figure 18: Volatility Workbench execution log displaying the extracted Process List

Note: The log accurately shows winpmem\_mini\_x (PID 19448) actively running, reflecting the acquisition tool executing at the time of the capture.

## 6. CONCLUSION

A complete digital forensics acquisition and analysis workflow was successfully executed via a combination of GUI and CLI tools. The target physical USB drive was securely imaged and verified using FTK Imager. Simultaneously, the system's live RAM was extracted via the Command Prompt using WinPmem. Finally, Volatility 3 command-line syntax was utilized to parse the raw memory file, successfully recovering critical live-state artifacts, including the process list and execution timestamps.